# APPENDIX - A
## Data Structures Lab

1. **Write a program to search for an element in an array using binary and linear search.**

```
/* Binary and Linear Search */
#include<stdio.h>
int main()
{
int i,x,n,ch,z=0;
int a[10],mid,high,low;
printf ("\n Binary and Linear Search \n");
printf ("1.Binary Search \n");
printf ("2.Linear Search \n");
printf ("\n Enter your choice:\n");
scanf("%d",&ch);
switch(ch)
{
case 1:
    printf("Enter total number of elements \n");
    scanf("%d",&n);
    printf("Enter elements in ascending order \n");
    for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    printf("Enter an element to be search  \n");
    scanf("%d",&x);
    low=0; high=n-1;
    while(low<=high)
    {
    mid=(low+high)/2;
    if(x<a[mid])
     high=mid-1;
    else if(x>a[mid])
     low=mid+1;
    else
    {
    printf("\n The element %d  present at position %d",x,mid+1);
    z=1;
    break;
    }
    }
    if(z==0)
```

```
    printf("\n Search element not present in the set");
    break;
case 2:
    printf("Enter total number of elements\n");
    scanf("%d",&n);
    printf("Enter the elements \n");
    for(i=0;i<n;i++)
    scanf("%d",&a[i]);
    printf("Enter an  element to be search \n");
    scanf("%d",&x);
    for(i=0;i<n;i++)
    {
    if(a[i]==x)
    {
    printf("\n The element %d present at position %d",x,i+1);
    z=1;
    break;
    }
    }
    if(z==0)
    printf("\n Element not present in the set");
    break;
default:
    printf("Invalid Choice");
}
return 0;
}
```

**Output**

```
Binary and Linear Search
1.Binary Search
2.Linear Search
Enter your choice:
1
Enter total number of elements
5
Enter elements in ascending order
10  20  30  40  50
Enter an element to be search
40
 The element 40  present at position 4
```

```
Binary and Linear Search
1.Binary Search
2.Linear Search
Enter your choice:
2
Enter total number of elements
5
Enter the elements
10  20  30  40  50
Enter an element to be search
30
 The element 30 present at position 3
```

**2. Write a program to sort list of N numbers using Bubble Sort algorithm.**

```
/* Bubble Sort */
#include<stdio.h>
int main()
{
int a[10],i,j,t,n;
printf(" Bubble Sort \n");
printf(" ---------- \n");
printf("Enter total number of elements \n");
scanf("%d",&n);
printf("Enter the elements \n");
for(i=0;i<n;i++)
scanf("%d",&a[i]);
for(i=0;i<n-1;i++)
{
for(j=0;j<n-1-i;j++)
{
if(a[j]>a[j+1])
{
t=a[j];
a[j]=a[j+1];
a[j+1]=t;
}
}
}
printf("\n The sorted elements are \n");
for(i=0;i<n;i++)
printf("\t %d",a[i]);
return 0;
}
```

**Output**

```
Bubble Sort
----------
Enter total number of elements
5
Enter the elements
10 30 20 50 40
The sorted elements are
        10      20      30      40      50
```

**3. Perform the Insertion and Selection Sort on the input {75,8,1,16,48,3,7,0} and display the output in descending order.**

```
/* Insertion and Selection Sort */
#include<stdio.h>
#define n 8
int main()
{
int a[n]={75,8,1,16,48,3,7,0};
int i,j,t,ch,min,loc;
printf("\n Input elements are: \n");
for(i=0;i<n;i++)
printf("%d\t",a[i]);
printf ("\n Insertion and Selection Sort\n");
printf ("1.Insertion Sort \n");
printf ("2.Selection Sort \n");
printf ("\n Enter your choice:\n");
scanf("%d",&ch);
switch(ch)
{
case 1:
    for(i=1;i<n;i++)
    {
    for(j=i;j>=1;j--)
    {
    if(a[j]>a[j-1])
    {
    t=a[j];
    a[j]=a[j-1];
    a[j-1]=t;
    }
    }
    }
    printf("\n Descending order using Insertion Sort \n");
    for(i=0;i<n;i++)
    printf("\t %d",a[i]);
    break;
case 2:
    for(i=0;i<n-1;i++)
    {
    min=a[i];
    loc=i;
    for(j=i+1;j<n;j++)
    {
```

```
    if(min<a[j])
    {
    min=a[j];
    loc=j;
    }
    }
    t=a[i];
    a[i]=min;
    a[loc]=t;
    }
    printf("\n Descending order using Selection Sort \n");
    for(i=0;i<n;i++)
    printf("\t %d",a[i]);
    break;
default:
    printf("\n Invalid Choice");
}
return 0;
}
```

**Output**

```
Input elements are:
75  8  1  16  48  3  7  0
Insertion and Selection Sort
1.Insertion Sort
2.Selection Sort
 Enter your choice:
1
 Descending order using Insertion Sort
75  48  16  8  7  3  1  0
```

```
Input elements are:
75  8  1  16  48  3  7  0
Insertion and Selection Sort
1.Insertion Sort
2.Selection Sort
 Enter your choice:
2
 Descending order using Selection Sort
75  48  16  8  7  3  1  0
```

4. **Write a program to insert the elements {61,16,8,27} into singly linked list and delete 8,61,27 from the list. Display your list after each insertion and deletion.**

```
/* Singly Linked List Operations */
#include<stdio.h>
#include <malloc.h>
#include <stdlib.h>
struct slist
{
int data;
struct slist *next;
};
typedef struct slist node;
```

```
node *head=NULL,*ptr,*newnode;
node *createnode();
void insert();
void del();
void display();
int ch,n=0,x=0,f=0,i;
int main()
{
while(1)
{
printf("\n --------Singly Linked List -------\n");
printf("\n 1.Insert");
printf("\n 2.Delete");
printf("\n 3.Display");
printf("\n 4.Exit");
printf("\n Enter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1:
    if(n>=4)
    printf("\n Given elements are Inserted");
    else
    {
    insert();
    printf("\n List created successfully");
    }
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
default:
    return 0;
}
}
}
void insert()
{
n++;
```

```
newnode=createnode();
if(head==NULL)
head=newnode;
else
{
ptr=head;
while(ptr->next!=NULL)
{
ptr=ptr->next;
}
ptr->next=newnode;
}}
node *createnode()
{
newnode = (node *) malloc(sizeof(node));
printf("\n Enter an element to be insert :");
scanf("%d", &newnode->data);
newnode->next=NULL;
return newnode;
}
void del()
{
node *prev;
ptr=head;
if(head==NULL)
{
printf("\n List is Empty");
}
else
{
printf("\n Enter an element to be delete :");
scanf("%d",&n);
if(head->data==n)
{
head=head->next;
free(ptr);
printf("\n Element deleted successfully");
}
else
{
while((ptr->data!=n) && (ptr!=NULL))
{
prev=ptr;
```

```
ptr=ptr->next;
}
if (ptr==NULL)
printf("\n Given element not found in the list ");
else
{
prev->next=ptr->next;
free(ptr);
printf("\n Element deleted sucessfuly");
}}}}
void display()
{
ptr=head;
printf("\n The contents of Singly Linked List are: \n");
if(head==NULL)
{
printf("\n Empty List");
}
else
{
while (ptr->next!=NULL)
{
printf("%d->", ptr->data);
ptr=ptr->next;
}
printf("%d->",ptr->data);
}
printf("NULL");
}
```

**Output**

```
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:1
Enter an element to be insert :61
List created successfully
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
```

```
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
61->NULL
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:1
Enter an element to be insert :16
List created successfully
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
61->16->NULL
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:1
Enter an element to be insert :8
List created successfully
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
61->16->8->NULL
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:1
Enter an element to be insert :27
```

```
List created successfully
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
61->16->8->27->NULL
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:2
Enter an element to be delete :8
Element deleted sucessfuly
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
61->16->27->NULL
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:2
Enter an element to be delete :61
Element deleted successfully
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
16->27->NULL
--------Singly Linked List -------
```

```
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:2
Enter an element to be delete :27
Element deleted sucessfuly
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:3
The contents of Singly Linked List are:
16->NULL
--------Singly Linked List -------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice:4
```

5. **Write a program to insert the elements {45, 34, 10, 63, 3} into linear queue and delete three elements from the list. Display your list after each insertion and deletion.**

```
/* Linear Queue Operations using Linked List */
#include<stdio.h>
#include<malloc.h>
#include<stdlib.h>
#define size  5
struct node
{
int info;
struct node *link;
}*head,*front,*rear,*ptr;
void  insert();
int  del();
void  display();
int ch,item,n=0,i;
int main()
{
while(1)
{
printf("\n Queue Operations");
```

```
printf("\n ----------------");
printf("\n 1.Insert");
printf("\n 2.Delete");
printf("\n 3.Display");
printf("\n 4.Exit");
printf("\n Enter your choice :");
scanf("%d",&ch);
switch(ch)
{
case 1:
    if (n>=size)
    printf("Queue is Full");
    else
    {
    insert();
    printf("\n List created successfully");
    }
    break;
case 2:
    del();
    break;
case 3:
    display();
    break;
default:
    return 0;
}
}
}
void insert()
{
printf("\n Enter an element to be insert:");
scanf("%d",&item);
ptr=(struct node *) malloc(sizeof(struct node));
ptr->info=item;
ptr->link=NULL;
if(rear==NULL)
{
front=ptr;
rear=ptr;
}
else
{
```

```
rear->link=ptr;
rear=ptr;
}
n++;
}
int del()
{
int n;
ptr=front;
if(front==NULL)
printf("\n Queue is Empty \n");
else
if(front==rear)
front=rear=NULL;
else
front=front->link;
n=ptr->info;
free(ptr);
printf("\n Deleted element is:%d \n",n);
return(0);
}
void display()
{
ptr=front;
printf("\n Queue elements are: \n");
if(front==NULL)
printf("\n Queue is Empty \n");
while(ptr!=NULL)
{
printf("%d->",ptr->info);
ptr=ptr->link;
}
printf("NULL");
}
```

**Output**

```
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
```

```
Enter your choice :1
Enter an element to be insert:45
List created successfully
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :3
Queue elements are:
45->NULL
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :1
Enter an element to be insert:34
List created successfully
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :3
Queue elements are:
45->34->NULL
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :1
Enter an element to be insert:10
List created successfully
Queue Operations
---------------
1.Insert
2.Delete
```

```
3.Display
4.Exit
Enter your choice :3
Queue elements are:
45->34->10->NULL
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :1
Enter an element to be insert:63
List created successfully
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :3
Queue elements are:
45->34->10->63->NULL
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :1
Enter an element to be insert:3
List created successfully
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :3
Queue elements are:
45->34->10->63->3->NULL
Queue Operations
---------------
```

```
1.Insert
 2.Delete
 3.Display
 4.Exit
 Enter your choice :1
Queue is Full
 Queue Operations
 ---------------
 1.Insert
 2.Delete
 3.Display
 4.Exit
 Enter your choice :2
 Deleted element is:45
 Queue Operations
 ---------------
 1.Insert
 2.Delete
 3.Display
 4.Exit
 Enter your choice :3
 Queue elements are:
34->10->63->3->NULL
 Queue Operations
 ---------------
 1.Insert
 2.Delete
 3.Display
 4.Exit
 Enter your choice :2
 Deleted element is:34
 Queue Operations
 ---------------
 1.Insert
 2.Delete
 3.Display
 4.Exit
 Enter your choice :3
 Queue elements are:
10->63->3->NULL
 Queue Operations
 ---------------
```

```
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :2
Deleted element is:10
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :3
Queue elements are:
63->3->NULL
Queue Operations
---------------
1.Insert
2.Delete
3.Display
4.Exit
Enter your choice :4
```

6. **Write a program to simulate the working of Circular queue using an array.**

```
/*Circular Queue Operations using Array*/
#include<stdio.h>
#include<ctype.h>
#include<stdlib.h>
#define max 5
void add(int n);
void del(int n);
void display();
int a[max];
int front=-1,rear=-1;
int ch,i,item;
int main()
{
while(1)
{
printf("\n Circular Queue Operations");
printf("\n ----------------------");
printf("\n 1.Insertion");
printf("\n 2.Deletion");
```

```
printf("\n 3.Display");
printf("\n 4.Exit");
printf("\n Enter Your choice :");
scanf("%d",&ch);
switch(ch)
{
case 1:
    add(max);
    break;
case 2:
    del(max);
    break;
case 3:
    display();
    break;
default:
    return 0;
}
}
}
void add(int n)
{
if((rear+1)>=n)
printf("\n Queue is Full");
else
{
printf("\n Enter the Element to be insert :");
scanf("%d",&item);
if(rear==-1)
front=rear=0;
else
rear=(rear+1)%n;
a[rear]=item;
printf("\n Rear=%d : Front=%d \n",rear,front);
}
}
void del(int n)
{
int x;
if(front==-1)
{
printf("\n Queue is Empty");
}
```

```
else
{
x=a[front];
if(front==rear)
front=rear=-1;
else
front=(front+1)%n;
printf("\n Deleted element is:%d ",x);
printf("\n Rear=%d : Front=%d \n",rear,front);
}
}
void display()
{
if(front==-1)
printf("\n Queue is empty");
else
{
printf("\n Queue Elements are\n");
for(i=front;i<=rear;i++)
printf("\t%d",a[i]);
}
}
```

**Output**

```
Circular Queue Operations
-------------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :1
Enter the Element to be insert :10
Rear=0 : Front=0
Circular Queue Operations
-------------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :1
Enter the Element to be insert :20
Rear=1 : Front=0
Circular Queue Operations
-------------------------
```

```
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :1
Enter the Element to be insert :30
Rear=2 : Front=0
Circular Queue Operations
-----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :1
Enter the Element to be insert :40
Rear=3 : Front=0
Circular Queue Operations
-----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :1
Enter the Element to be insert :50
Rear=4 : Front=0
Circular Queue Operations
-----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :1
Queue is Full
Circular Queue Operations
-----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :3
Queue Elements are
     10     20     30     40     50
Circular Queue Operations
-----------------------
```

```
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :2
Deleted element is:10
Rear=4 : Front=1
Circular Queue Operations
----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :3
Queue Elements are
      20     30     40     50
Circular Queue Operations
----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :2
Deleted element is:20
Rear=4 : Front=2
Circular Queue Operations
----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :3
Queue Elements are
      30     40     50
Circular Queue Operations
----------------------
1.Insertion
2.Deletion
3.Display
4.Exit
Enter Your choice :4
------------------------------
Process exited after 40.05 seconds with return value 0
Press any key to continue . . .
```

7. **Write a program to insert the elements {61,16,8,27} into ordered singly linked list and delete 8,61,27 from the list. Display your list after each insertion and deletion.**

**(Refer. Program: 4)**

8. **Write a program for Tower of Honoi problem using recursion.**

```
/* Towers of Hanoi using Recursion */
#include<stdio.h>
void towers(int n, char sr, char ax, char dt);
int main()
{
 int n;
 printf("Towers of Hanoi \n");
 printf("---------- \n");
 printf("Enter number of disks \n");
 scanf("%d",&n);
 towers(n, 'S', 'A', 'D');
 return 0;
}
void towers(int n, char sr, char ax, char dt)
{
 if (n==1)
 printf("\n Move disk 1 from tower %c to tower %c", sr, dt);
 else
 {
  towers(n-1, sr, dt, ax);
  printf("\n Move disk %d from tower %c to tower %c", n, sr, dt);
  towers(n-1, ax, sr, dt);
 }
}
```

**Output**

```
Towers of Hanoi
----------
Enter number of disks
3
 Move disk 1 from tower S to tower D
 Move disk 2 from tower S to tower A
 Move disk 1 from tower D to tower A
 Move disk 3 from tower S to tower D
 Move disk 1 from tower A to tower S
 Move disk 2 from tower A to tower D
 Move disk 1 from tower S to tower D
```

**9. Write recursive program to find GCD of 3 numbers.**

```
/* GCD of three Numbers */
#include<stdio.h>
int gcd(int a, int b);
int main()
{
 int a,b,c,d;
 printf(" GCD of Three Numbers \n");
 printf(" ---------- \n");
 printf("Enter three numbers \n");
 scanf("%d%d%d",&a,&b,&c);
 d=gcd(a,gcd(b,c));
 printf("\n GCD of %d  %d  and %d is %d",a,b,c,d);
 return 0;
 }
int gcd(int a, int b)
{
 if (b==0)
    return(a);
 else if (a<b)
   return(gcd(b,a));
 else
   return(gcd(b,a%b));
  }
```

**Output**

```
GCD of Three Numbers
 ------------------------
Enter three numbers
12  18  24
 GCD of 12  18  and 24 is 6
```

**10. Write a program to demonstrate working of stack using linked list.**

```
/*Stack Operations using Linked List*/
#include<stdio.h>
#include<stdlib.h>
#define max 5
struct node
{
int info;
struct node *link;
}*top,*ptr;
void push(int num);
int pop();
```

```
void peek();
void display();
int data,ch,x=1;
int main()
{
while(1)
{
printf("\n Stack Operations");
printf("\n ----------");
printf("\n 1.Push");
printf("\n 2.Pop");
printf("\n 3.Peek");
printf("\n 4.Display");
printf("\n 5.Exit");
printf("\n Enter your choice : ");
scanf("%d",&ch);
switch(ch)
{
case 1: if (x<=max)
      {
      printf("\n Enter an element to be insert:");
      scanf("%d",&data);
      push(data);
      x++;
      }
      else
      printf("\n Stack Full");
      break;
case 2: pop();
       break;
case 3: peek();
       break;
case 4: printf("\n Elements of the stack are: \n");
      display();
      break;
default: return 0;
}
}
}
void push(int num)
{
ptr=malloc(sizeof(struct node));
ptr->info=num;
```

```
ptr->link=NULL;
if(top==NULL)
top=ptr;
else
{
ptr->link=top;
top=ptr;
}
}
int pop()
{
int n;
ptr=top;
if(ptr==NULL)
{
printf("\n Stack is empty \n");
return 0;
}
top=top->link;
n=ptr->info;
printf("\n Popped element is %d \n",n);
free(ptr);
return;
}
void peek()
{
ptr=top;
if(ptr==NULL)
{
printf("\n Stack is empty");
return;
}
else
printf("\nTop element of the Stack is %d ",ptr->info);
}
void display()
{
ptr=top;
if(ptr==NULL)
{
printf("\n stack is empty");
return;
}
```

```
else
{
while(ptr!=NULL)
{
printf("\n%d ",ptr->info);
ptr=ptr->link;
}
}
}
```

**Output**

```
Stack Operations
----------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 1
Enter an element to be insert:10
Stack Operations
----------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 1
Enter an element to be insert:20
Stack Operations
----------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 1
Enter an element to be insert:30
Stack Operations
----------------
1.Push
2.Pop
3.Peek
```

```
4.Display
5.Exit
Enter your choice : 1
Enter an element to be insert:40
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 1
Enter an element to be insert:50
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 1

Stack Full
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 4
Elements of the stack are:
50
40
30
20
10
 Stack Operations
 ---------------
 1.Push
 2.Pop
 3.Peek
```

```
4.Display
5.Exit
Enter your choice : 3
Top element of the Stack is 50
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 2
Popped element is 50
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 2
Popped element is 40
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 4
Elements of the stack are:
30
20
10
Stack Operations
---------------
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice : 5
```

**11. Write a program to convert an infix expression x^y/(5*z)+2 to its postfix expression.**

```
/* Infix to Postfix Conversion */
#include<stdio.h>
#include<ctype.h>
char stack[20];
int top=-1;
char push(char e);
char pop();
int pr(char e);
int main()
{
char post[20],ch;
char infix[]="x^y/(5*z)+2";
int i=0,k=0;
printf(" Infix Expression: ");
printf("%s",infix);
push('#');
while ((ch=infix[i++])!='\0')
{
if(ch=='(')
push(ch);
else if(isalnum(ch))
post[k++]=ch;
else if(ch== ')')
{
while (stack[top]!='(')
post[k++]=pop();
}
else
{
while (pr(stack[top])>=pr(ch))
post[k++]=pop();
push(ch);
}
}
while (stack[top]!='#')
post[k++]=pop();
post[k]='\0';
printf("\n Postfix Expression: %s\n",post);
}
char push(char e)
{
stack[++top]=e;
```

```
return;
}
char pop()
{
return(stack[top--]);
}
int pr(char e)
{
switch(e)
{
case '#': return 0;
case '(': return 1;
case '+':
case '-': return 2;
case '*':
case '/':
case '%': return 3;
}
return 1;
}
```

**Output**

```
Infix Expression: x^y/(5*z)+2
 Postfix Expression: xy5z*2+(/^
```

**12. Write a program to evaluate a postfix expression 5 3 + 8 2 - ***

```
/* Evaluate a postfix expression */
#include<stdio.h>
#include<ctype.h>
int stack[20];
int top=-1;
int push(int x);
int pop();
int main()
{
char exp[8]="53+82-*";
char *e;
int n1,n2,n3,num;
printf("Input the expression: ");
printf("%s",exp);
e=exp;
while(*e!='\0')
{
if(isdigit(*e))
```

```
{
num=*e-48;
push(num);
}
else
{
n1=pop();
n2=pop();
switch(*e)
{
case '+':
    n3=n1+n2;
    break;
case '-':
    n3=n2-n1;
    break;
case '*':
    n3=n1*n2;
    break;
case '/':
    n3=n2/n1;
    break;
    }
push(n3);
}
e++;
}
printf("\n The result of expression=%d",pop());
return 0;
}
int push(int x)
{
stack[++top]=x;
return 0;
}
int pop()
{
return stack[top--];
}
```

**Output**

```
Input the expression: 53+82-*
The result of expression=48
```

**13. Write a program to create a binary tree with the elements {18,15,40,50,30,17,41} after creation insert 45 and 19 into tree and delete 15,17 and 41 from tree. Display the tree on each insertion and deletion operation.**

```
/* Binary Tree Operations */
#include <stdio.h>
#include <stdlib.h>
#include<malloc.h>
typedef struct node
{
int data;
struct node* left;
struct node* right;
} node;
node *create(int data);
void insert(node** root, int data);
node *getrightnode(node* root);
void deleterightnode(node* root, node* dnode);
void del (node** root, int data);
void inorder(node* root);
int main()
{
node* root = NULL;
int key;
insert(&root,18);
insert(&root,15);
insert(&root,40);
insert(&root,50);
insert(&root,30);
insert(&root,17);
insert(&root,41);
printf("Inorder traversal of the given Binary Tree is: ");
inorder(root);
printf("\n");
key = 45;
insert(&root, key);
printf("After Insertion of %d: ", key);
inorder(root);
printf("\n");
key = 19;
insert(&root, key);
printf("After Insertion of %d: ", key);
inorder(root);
```

```
printf("\n");
key = 15;
del (&root, key);
printf("After Deletion of %d: ", key);
inorder(root);
printf("\n");
key = 17;
del (&root, key);
printf("After Deletion of %d: ", key);
inorder(root);
printf("\n");
key = 41;
del (&root, key);
printf("After Deletion of %d: ", key);
inorder(root);
printf("\n");
return 0;
}
node* create(int data)
{
node* newnode = (node*)malloc(sizeof(node));
newnode->data = data;
newnode->left = NULL;
newnode->right = NULL;
return newnode;
}
void insert(node** root, int data)
{
node* newnode = create(data);
node* temp;
node* q[100];
int f = -1, r = -1;
if (*root == NULL)
{
*root = newnode;
return;
}
q[++r] = *root;
while (f != r) {
temp = q[++f];
if (temp->left == NULL)
{
```

```
temp->left = newnode;
return;
}
else
{
q[++r] = temp->left;
}
if (temp->right == NULL)
{
temp->right = newnode;
return;
}
else
{
q[++r] = temp->right;
 }
 }
}
node* getrightnode(node* root)
{
 node* temp;
 node* q[100];
 int f = -1, r = -1;
 q[++r] = root;
 while (f != r) {
 temp = q[++f];
 if (temp->left != NULL)
{
 q[++r] = temp->left;
 }
 if (temp->right != NULL)
{
 q[++r] = temp->right;
 }
 }
 return temp;
}
void deleterightnode(node* root, node* dnode)
{
 node* temp;
 node* q[100];
 int f = -1, r = -1;
 q[++r] = root;
```

```
 while (f != r)
{
temp = q[++f];
if (temp == dnode)
{
temp = NULL;
free(dnode);
return;
}
if (temp->right != NULL)
{
if (temp->right == dnode)
{
temp->right = NULL;
free(dnode);
return;
}
else
{
q[++r] = temp->right;
}
}
if (temp->left != NULL)
{
if (temp->left == dnode)
{
temp->left = NULL;
free(dnode);
return;
}
else
{
q[++r] = temp->left;
}
}
}
}
void del (node** root, int data)
{
node* temp;
node* keynode = NULL;
node* q[100];
int f = -1, r = -1;
```

```
q[++r] = *root;
if (*root == NULL)
{
 printf("Tree is empty.\n");
 return;
}
if ((*root)->left == NULL && (*root)->right == NULL)
{
if ((*root)->data == data)
{
free(*root);
*root = NULL;
return;
}
else
{
printf("node not found.\n");
return;
 }
 }

 while (f != r) {
temp = q[++f];
if (temp->data == data)
{
keynode = temp;
}
if (temp->left != NULL)
{
q[++r] = temp->left;
}
if (temp->right != NULL)
{
q[++r] = temp->right;
  }
  }
if (keynode != NULL)
{
node* deepestnode = getrightnode(*root);
keynode->data = deepestnode->data;
deleterightnode(*root, deepestnode);
}
else
```

```
printf("node not found.\n");
}
void inorder(node* root)
{
if (root == NULL)
{
return;
}
inorder(root->left);
printf("%d ", root->data);
inorder(root->right);
}
```

**Output**

```
Inorder traversal of the given Binary Tree is: 50 15 30 18 17 40 41
After Insertion of 45: 45 50 15 30 18 17 40 41
After Insertion of 19: 45 50 19 15 30 18 17 40 41
After Deletion of 15: 45 50 19 30 18 17 40 41
After Deletion of 17: 50 19 30 18 45 40 41
After Deletion of 41: 50 19 30 18 45 40
```

**14. Write a program to create binary search tree with the elements {2,5,1,3,9,0,6} and perform inorder, preorder and postorder traversal.**

```
/* Binary Search Tree Traversal */
#include <stdio.h>
#include <stdlib.h>
struct btnode
{
int data;
struct btnode *left;
struct btnode *right;
}*root=NULL,*temp=NULL,*t2,*t1;
void create();
void search(struct btnode *t);
void inorder(struct btnode *t);
void preorder(struct btnode *t);
void postorder(struct btnode *t);
int main()
{
int ch;
printf("\n Binary Search Tree");
printf("\n ----------------");
printf("\n 1.Create Binary Tree");
```

```
printf("\n 2.Inorder Traversal");
printf("\n 3.Preorder Traversal");
printf("\n 4.Postorder Traversal");
printf("\n 5.Exit");
while(1)
{
printf("\n Enter your choice : ");
scanf("%d", &ch);
switch (ch)
{
case 1:
     create();
     break;
case 2:
     printf(" Inorder \n");
     printf(" ------- \n");
     inorder(root);
     break;
case 3:
     printf(" Preorder \n");
     printf(" -------- \n");
     preorder(root);
     break;
case 4:
     printf(" Postorder \n");
     printf(" --------- \n");
     postorder(root);
     break;
default:
     return 0;
}
}
}
void create()
{
int i,a[7]={2,5,1,3,9,0,6};
for(i=0;i<7;i++)
{
temp=(struct btnode *)malloc(1*sizeof(struct btnode));
temp->data = a[i];
temp->left=temp->right=NULL;
if (root == NULL)
root = temp;
```

```
else
search(root);
}
printf("Tree created successfully");
}
void search (struct btnode  *t)
{
if ((temp->data > t->data) && (t->right != NULL))
search(t->right);
else if ((temp->data > t->data) && (t->right == NULL))
t->right=temp;
else if ((temp->data < t->data) && (t->left != NULL))
search(t->left);
else if ((temp->data < t->data) && (t->left == NULL))
t->left=temp;
}
void inorder (struct btnode *t)
{
if (root==NULL)
{
printf("\n No elements in a tree to display");
return;
}
if (t->left!=NULL)
inorder(t->left);
printf("\t %d", t->data);
if (t->right != NULL)
inorder(t->right);
}
void preorder (struct btnode *t)
{
if (root==NULL)
{
printf("\n No elements in a tree to display");
}
printf("\t %d", t->data);
if (t->left!=NULL)
preorder(t->left);
if (t->right != NULL)
preorder(t->right);
}
void postorder(struct btnode *t)
{
```

```
if (root==NULL)
{
printf("\n No elements in a tree to display");
return;
}
if (t->left!=NULL)
postorder(t->left);
if (t->right!=NULL)
postorder(t->right);
printf("\t %d", t->data);
}
```

**Output**

```
Binary Search Tree
------------------
1.Create Binary Tree
2.Inorder Traversal
3.Preorder Traversal
4.Postorder Traversal
5.Exit
Enter your choice : 1
Tree created successfully
Enter your choice : 2
Inorder
-------
     0     1     2     3     5     6     9
Enter your choice : 3
Preorder
--------
     2     1     0     5     3     9     6
Enter your choice : 4
Postorder
---------
     0     1     3     6     9     5     2
Enter your choice :5
```

**15. Write a program to Sort the following elements using heap sort {9.16,32,8,4,1,5,8,0}.**

```
/* Heap Sort */
#include<stdio.h>
void heap(int a[], int n, int i);
void heapsort(int a[], int n);
int main()
{
```

```
int a[]={9,16,32,8,4,1,5,8,0};
int i,n=9;
printf("\n Input elements are :\n");
for (i=0;i<n;i++)
printf("%d\t",a[i]);
heapsort(a, n);
printf("\n Sorted elements are:\n");
for (i=0;i<n;i++)
printf("%d\t",a[i]);
return 0;
}
void heap(int a[],int n,int i)
{
int t,max,left,right;
max=i;
right=2*i+2;
left=2*i+1;
if (left<n && a[left]>a[max])
max = left;
if (right<n && a[right]>a[max])
max=right;
if (max!=i)
{
t=a[i];
a[i]=a[max];
a[max]=t;
heap(a, n, max);
}
}
void heapsort(int a[], int n)
{
int i,t;
for (i=n/2-1;i>=0;i--)
heap(a,n,i);
for (i=n-1;i>0;i--)
{
t=a[0];
a[0]=a[i];
a[i]=t;
heap(a,i,0);
}
}
```

**Output**

```
Input elements are :
9    16    32    8    4    1    5    8    0
Sorted elements are:
0    1    4    5    8    8    9    16    32
```

**16. Given S1={"Flowers"} ; S2={"are beautiful"}**

**I. Find the length of S1**

**II. Concatenate S1 and S2**

**III. Extract the substring "low" from S1.**

**IV. Find "are" in S2 and replace it with "is".**

```
/* String Operations */
#include<stdio.h>
#include<string.h>
void length(char *str);
void concat(char *str1, char *str2);
void extract(char *str,int pos, int e);
void replace(char *str, char *rstr, int pos);
int main()
{
int len,ch,pos,e;
while(1)
{
char s1[20]="Flowers";
char s2[20]=" are beautiful";
printf("\n==================");
printf("\n s1 = %s   s2 = %s\n",s1,s2);
printf("\n String Operations ");
printf("\n==================");
printf("\n 1.Length");
printf("\n 2.Concatenate");
printf("\n 3.Substring");
printf("\n 4.Replace");
printf("\n 5.Exit");
printf("\n Enter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1:
        length(s1);
        break;
case 2:
        concat(s1,s2);
        break;
```

```
case 3:
        extract(s1,1,3);
        break;
case 4:
        replace(s2,"is ",0);
        break;
default:
       return 0;
}
}
}
void length(char *str)
{
int i=0, len=0;
while(str[i]!='\0')
{
len++;
i++;
}
printf("\n Length of %s = %d",str,len);
}
void concat(char str1[20], char str2[20])
{
int i=0,j=0;
while(str1[i]!='\0')
i++;
while(str2[j]!='\0')
{
str1[i]=str2[j];
i++;
j++;
}
str1[i]='\0';
printf("\n Concatenated string = %s",str1);
}
void extract(char *str,int pos, int e)
{
int  i=0,j=0;
char substr[10];
for(i=pos;i<=e;i++)
{
substr[j]=str[i];
 j++;
```

```
}
substr[j]='\0';
printf("\n Substring = %s",substr);
}
void replace(char *str, char *rstr, int pos)
{
char z[25];
int i=0,j=0,k=0;
for(i=0;i<strlen(str);i++)
{
if(i==pos)
{
for(k=pos;k<strlen(rstr);k++)
{
z[j]=rstr[k];
j++;
i++;
}
}
else
{
z[j]=str[i];
j++;
}
}
z[j]='\0';
printf("\n Output = %s",z);
}
```

**Output**

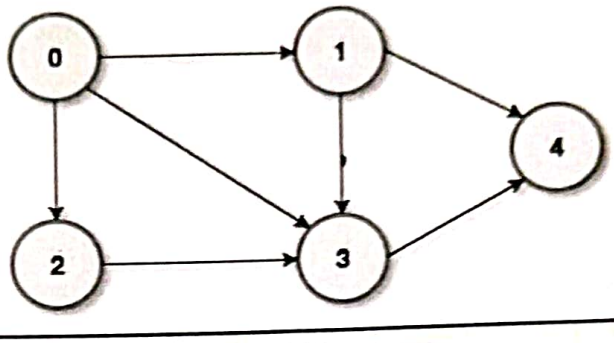
```
==================
 s1 = Flowers   s2 =  are beautiful
 String Operations
==================
 1.Length
 2.Concatenate
 3.Substring
 4.Replace
 5.Exit
 Enter your choice:1
 Length of Flowers = 7
==================
```

```
s1 = Flowers   s2 =  are beatiful
String Operations
==================
1.Length
2.Concatenate
3.Substring
4.Replace
5.Exit
Enter your choice:2
Concatenated string = Flowers are beautiful.
==================
s1 = Flowers   s2 =  are beautiful
String Operations
==================
1.Length
2.Concatenate
3.Substring
4.Replace
5.Exit
Enter your choice:3
Substring = low
==================
s1 = Flowers   s2 =  are beautiful
String Operations
==================
1.Length
2.Concatenate
3.Substring
4.Replace
5.Exit
Enter your choice:4

Output = is  beatiful
==================
s1 = Flowers   s2 =  are beautiful
String Operations
==================
1.Length
2.Concatenate
3.Substring
4.Replace
5.Exit
Enter your choice:5
```

**17. Write a program to implement adjacency matrix of a graph.**

**Graph:**




```
/* Adjacency Matrix Representation of Graph */
#include<stdio.h>
#define v 5
void init(int a[][v]);
void addedge(int a[][v],int src, int dest);
void display(int a[][v]);
int main()
{
int b[v][v];
init(b);
addedge(b,0,1);
addedge(b,0,2);
addedge(b,0,3);
addedge(b,1,3);
addedge(b,1,4);
addedge(b,2,3);
addedge(b,3,4);
display(b);
return 0;
}
void init(int a[][v])
{
int i,j;
for(i = 0; i < v; i++)
for(j = 0; j < v; j++)
a[i][j] = 0;
}
void addedge(int a[][v],int src, int dest)
{
a[src][dest] = 1;
}
void display(int a[][v])
{
```

```
int i, j;
printf("\n Adjacency Matrix of Given graph is:\n");
printf("\n==============================\n");
for(i = 0; i < v; i++)
{
for(j = 0; j < v; j++)
{
 printf("%d\t ", a[i][j]);
 }
 printf("\n");
}
}
```

**Output**

```
Adjacency Matrix of Given graph is:
==============================
0    1    1    1    0
0    0    0    1    1
0    0    0    1    0
0    0    0    0    1
0    0    0    0    0
```

**18. Write a program to insert/retrieve an entry into hash/ from a hash table with open addressing using linear probing.**

```
/* Hash Table */
#include<stdio.h>
#include<stdlib.h>
#include<malloc.h>
struct item
{
int key;
int value;
};
struct hashtable
{
 int flag;
 struct item *data;
};
struct hashtable *a;
int size = 0;
int max = 10;
```

```
void init_a();
int hashcode(int key);
int insert(int key, int value);
void del(int key);
int size_of_hashtable();
void display();
int main()
{
int ch, key, value, n, c,i;
a=(struct hashtable*) malloc(max * sizeof(struct hashtable*));
while (1)
{
printf("Hash Table with Linear Probing");
printf("\n======================");
printf("\n 1.Inserting item to the Hash Table");
printf("\n 2.Removing item from the Hash Table");
printf("\n 3.Check the size of Hash table");
printf("\n 4.Display Hash table");
printf("\n 5.Exit");
printf("\n Enter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1:
    printf("Enter key and value-:\t");
    scanf("%d %d", &key, &value);
    insert(key, value);
    break;
case 2:
    printf("\n Enter the key to delete:");
    scanf("%d", &key);
    del(key);
    break;
case 3:
    n=size_of_hashtable();
    printf("Size of Hash table is-:%d\n", n);
    break;
case 4:
    display();
    break;
default:
    return 0;
```

```
}
}
}
void init_a()
{
int i;
for (i=0;i<max;i++)
{
a[i].flag=0;
a[i].data=NULL;
printf ("OK");
}
}
int hashcode(int key)
{
return (key%max);
}
int insert(int key, int value)
{
int index=hashcode(key);
int i=index;
struct item *new_item = (struct item*) malloc(sizeof(struct item));
new_item->key=key;
new_item->value=value;
while (a[i].flag==1)
{
if (a[i].data->key==key)
{
printf("\n Key already exists, hence updating its value \n");
a[i].data->value=value;
return 0;
}
i=(i+1)%max;
if (i==index)
{
printf("\n Hash table is full, cannot insert any more item \n");
return 0;
}
 }
a[i].flag = 1;
a[i].data = new_item;
size++;
```

```
printf("\n Key (%d) has been inserted \n", key);
return 0;
}
void del(int key)
{
 int index = hashcode(key);
 int  i = index;
while (a[i].flag != 0)
{
if (a[i].flag == 1  &&  a[i].data->key == key )
{
a[i].flag =  2;
a[i].data = NULL;
size--;
printf("\n Key (%d) has been removed \n", key);
return;
}
i = (i + 1) % max;
if (i == index)
break;
}
printf("\n This key does not exist \n");
}
void display()
{
int i;
for (i = 0; i < max; i++)
{
struct item *current = (struct item*) a[i].data;
if (current == NULL)
{
printf("\n Array[%d] has no elements \n", i);
}
else
{
 printf("\n Array[%d] has elements -: \n  %d (key) and %d(value) ", i, current-
    >key, current->value);
}
}
}
```

```
int size_of_hashtable()
{
return size;
}
```

**Output**

```
Hash Table with Linear Probing
======================
 1.Inserting item to the Hash Table
 2.Removing item from the Hash Table
 3.Check the size of Hash table
 4.Display Hash table
 5.Exit
Enter your choice:1
Enter key and value-:   1 20
 Key (1) has been inserted
Hash Table with Linear Probing
======================
 1.Inserting item to the Hash Table
 2.Removing item from the Hash Table
 3.Check the size of Hash table
 4.Display Hash table
 5.Exit
 Enter your choice:1
Enter key and value-:   2 25
 Key (2) has been inserted
Hash Table with Linear Probing
======================
 1.Inserting item to the Hash Table
 2.Removing item from the Hash Table
 3.Check the size of Hash table
 4.Display Hash table
 5.Exit
 Enter your choice:3
Size of Hash table is-:2
Hash Table with Linear Probing
======================
 1.Inserting item to the Hash Table
 2.Removing item from the Hash Table
 3.Check the size of Hash table
 4.Display Hash table
 5.Exit
 Enter your choice:4
```

```
Array[0] has elements -:
 9989232 (key) and 0(value)
Array[1] has elements -:
 1 (key) and 20(value)
Array[2] has elements -:
 2 (key) and 25(value).
```